ASKBASE

From Questions to Verified SQL Insights

AI Agent Development Report

Detailed work allocation for four parallel engineering workstreams

Product goal	Turn natural-language questions about unfamiliar or messy databases into verified SQL-backed answers, result tables, charts and transparent warnings.
Repository	github.com/aadityajoshi01/AskBase-From-questions-to-verified-SQL-insights
Development approach	Inspect and reuse existing code first. The structure in this report is a target architecture, not a claim about the repository's current files.

This document contains the application overview, architecture, detailed responsibilities, interfaces, testing expectations and integration plan for all four agents. It is a technical work report, not a set of prompts.

1. Application Overview

AskBase is an AI-powered data analyst. A user connects a database, asks a question in plain English, and receives an answer grounded in SQL execution. The system should work with unfamiliar schemas and expose data-quality issues rather than assume that data is clean.

Core user journey

1. **Connect** — Select or configure a supported database.
2. **Discover** — Inspect tables, columns, data types, keys and relationships.
3. **Profile** — Identify missing values, duplicate-like records, inconsistent categories, type problems and suspicious values.
4. **Understand** — Interpret the natural-language question against the actual schema and relevant quality warnings.
5. **Generate** — Generate SQL using the configured LLM and schema context.
6. **Validate** — Parse and validate the SQL before any execution.
7. **Execute** — Run the approved query with read-only permissions, timeouts and result limits where supported.
8. **Verify** — Check that the answer follows from actual results and disclose limitations.
9. **Present** — Show the explanation, SQL, result table, relevant chart and warnings.

Expected user experience

- A question input that supports arbitrary user questions, not only hardcoded examples.
- A visible SQL section so users can inspect how an answer was derived.
- A result table and a chart only when the returned data makes a chart meaningful.
- Clear states for success, clarification needed, empty results, unsupported questions, execution errors and model unavailability.
- Warnings for sampled data, truncated results, missing values and other limitations that affect interpretation.
- No fabricated result values and no successful-answer presentation when SQL execution or verification fails.

Development environment and assumptions

The team has reported a Windows development environment and a local Qwen3-8B Q4_K_M model served by Llamafile through an OpenAI-compatible endpoint. Treat the endpoint and model identifier as configuration. Confirm the actual endpoint, dependencies and existing app framework in the repository before changing code. SQLite is a suitable reproducible test database if it fits the existing project.

Repository status note

The GitHub integration returned 404 when repository metadata was requested. This does not establish whether the repository is private, renamed, mistyped or otherwise inaccessible. The agents should confirm access and inspect the actual source tree before implementing against specific filenames.

2. Target Architecture

The UI orchestrates the workflow. Database discovery and profiling supply context to the AI. AI-generated SQL passes through a safety boundary before execution. Verification and answer synthesis use the actual query result.

Layer	Responsibility	Owner
UI / orchestration	Question input, connection selection, loading and error states, answer/table/chart rendering, end-to-end flow	Agent 4
Schema discovery	Tables, columns, types, keys, relationships and compact schema context	Agent 2
Data profiling	Missingness, duplicates, category inconsistencies, type/date problems, sample metadata	Agent 2
AI / SQL generation	Question interpretation, schema-aware SQL generation, bounded repair, answer synthesis	Agent 1
SQL validation	Dialect-aware parsing, read-only allowlist, multi-statement rejection and unsafe-operation checks	Agent 3
Execution / verification	Restricted execution, timeout/row limits, actual result metadata, answer evidence checks	Agent 3

End-to-end flow

User question	Schema + profile	AI-generated SQL
Validated SQL	Read-only execution	Verified result
Answer + table + chart + warnings		

Suggested module layout

Use the equivalent existing files if they already exist; this is a proposed organization only.

```
AskBase/
■■■ app.py
■■■ backend/
■ ■■■ llm_client.py
■ ■■■ agent.py
■ ■■■ sql_generator.py
■ ■■■ schema_discovery.py
■ ■■■ data_profiler.py
■ ■■■ sql_validator.py
■ ■■■ query_executor.py
■ ■■■ result_verifier.py
■■■ tests/
■■■ sample_data/create_sample_db.py
■■■ requirements.txt
■■■ .env.example
■■■ INTEGRATION_CONTRACT.md
■■■ README.md
```

3. Agent 1 — AI Analyst and SQL Generation

Branch: feature/ai-agent

Mission

Implement the intelligence layer that interprets a natural-language question, uses the supplied schema and data-quality context, generates SQL and produces an explanation grounded in executed results. The module must not execute SQL independently or bypass the safety layer.

Work areas

- **LLM client:** reuse or improve the existing OpenAI-compatible client; configure base URL, model identifier, timeout and generation settings via environment variables; handle offline server, HTTP errors, timeouts, malformed responses and empty output.
- **Schema-aware generation:** use only the provided tables and columns; consider relationships and data types; support appropriate filters, sorting, joins and aggregations; avoid invented columns, currencies, units and time periods.
- **Clarification:** identify material ambiguity and return a clarification-needed state rather than silently choosing a consequential interpretation.
- **Structured output:** return SQL, assumptions, clarification status and a clarification question using the shared contract.
- **Bounded repair:** permit at most two repair attempts after initial generation for repairable SQL errors; every repaired query must go through the same validator and executor.
- **Answer synthesis:** use actual returned rows and columns; explain empty results and limitations; never invent values or claim failed execution succeeded.

Conceptual interfaces

```
generate_sql(question: str, schema: dict) -> dict
synthesize_answer(question: str, sql: str, result: dict) -> dict
```

Testing and deliverables

- At least 15 mocked-LLM tests covering filters, joins, aggregation, ambiguity, invalid schema, malformed output, offline server, timeout, repair limits, empty results and insufficient evidence.
- AI module implementation, prompt/configuration documentation, sample inputs and outputs, test results and known limitations.
- Tests must run without a live model or GPU.

Acceptance: Given a question and schema, the module produces structured SQL-generation output, handles model failures and never executes generated SQL directly.

4. Agent 2 — Database Intelligence and Data Quality

Branch: feature/data-profiling

Mission

Implement schema discovery and data-quality profiling so AskBase can understand unfamiliar databases and disclose data issues without silently modifying source records.

Work areas

- **Schema discovery:** discover tables and views where supported; collect column names, database types, nullability, primary keys and foreign-key relationships; create a compact schema summary for the AI.
- **Data profiling:** calculate missing-value counts/rates, duplicate-row and potential duplicate-key indicators, distinct-value counts, category frequencies, inconsistent whitespace/casing, unexpected types, numeric ranges and date parsing failures.
- **Interpretation:** label outliers as candidates for review, not automatically as errors; document metric denominators and how null/invalid values are treated.
- **Bounded processing:** use configurable samples or limits for large tables, label sample-based metrics and avoid unnecessary full-table memory loads.
- **Source integrity:** do not automatically delete duplicates, fill nulls, rename categories or alter source data; use read-only access where practical.
- **Synthetic dataset:** create a reproducible multi-table test database such as customers, orders, order_items and products with documented, intentional data-quality anomalies.

Conceptual interfaces

```
get_schema(database_config: dict) -> dict
profile_database(database_config: dict) -> dict
```

Testing and deliverables

- At least 12 tests for relationships, empty databases/tables, nulls, duplicates, mixed types, category inconsistencies, invalid dates, sampling, read-only operation and connection errors.
- Schema-discovery module, profiler, database creation script, tests, example reports and setup instructions.

Acceptance: AskBase can connect to the supported test database and produce a correct schema summary and bounded data-quality report without changing source data.

5. Agent 3 — SQL Safety, Execution and Verification

Branch: feature/sql-verification

Mission

Implement the safety boundary between model-generated SQL and the database. All SQL must be validated before execution, and all answers must reflect actual execution results.

Work areas

- **Dialect-aware validation:** use SQLGlot or an equivalent parser; permit explicitly supported read-only query constructs; reject writes, DDL, transaction-control operations, multiple statements and unsafe dialect-specific operations.
- **Layered controls:** use read-only database permissions or connections where possible, plus validation, execution timeouts, row limits and bounded payloads. A parser allowlist alone is not a complete security boundary.
- **Safe execution:** distinguish syntax errors, permission failures, timeouts, empty results and successful results; avoid unsafe value interpolation and never leak credentials or sensitive internal details.
- **Result verification:** return actual columns and rows; check that reported counts, totals and rankings match results; disclose truncation and limitations; avoid unsupported statistical or causal claims.
- **Single execution path:** ensure the UI and any retry/repair flow cannot bypass validation; revalidate every repaired query.

Conceptual interfaces

```
validate_sql(sql: str, dialect: str) -> dict
execute_query(sql: str, database_config: dict) -> dict
verify_result(question: str, result: dict) -> dict
```

Testing and deliverables

- At least 15 normal, negative and adversarial tests covering SELECT, joins, aggregation, subqueries/CTEs if supported, writes, DDL, multiple statements, unsafe functions, timeouts, row limits, empty results, permissions and answer/result mismatch.
- Safety/execution implementation, verification logic, tests, supported SQL documentation and test results.

Acceptance: No generated query reaches execution without validation. Dangerous queries are rejected, limits are enforced where supported, and failed execution cannot be presented as a successful insight.

6. Agent 4 — UI, Evaluation and Integration

Branch: feature/ui-evaluation

Mission

Deliver the usable AskBase application and integrate the other three modules into a single tested workflow. Preserve the existing UI framework and working architecture unless a documented technical reason requires change.

Work areas

- **Initial audit:** run the existing app, document the current entry point, dependencies, routes, database flow and reusable components.
- **Workspace:** provide database selection/connection, a natural-language question input, submit/loading states, schema-aware question suggestions and optional data-quality warnings.
- **Answer view:** show evidence-backed answer, SQL in an expandable section, results table, appropriate chart, execution/verification status, warnings and limitations.
- **Error handling:** support clarification, empty results, model offline, timeouts, invalid SQL and missing chart data without crashing; avoid stale results being shown for a new question.
- **Mocks:** make UI development possible before the LLM and other modules are fully integrated.
- **Evaluation:** create a reproducible question set with expected outcomes; measure execution success, result correctness, answer correctness, safety failures and latency with explicit denominators.
- **Integration:** maintain the contract, merge tested changes incrementally, run regression/end-to-end tests and verify clean setup.

Required evaluation scenarios

- Filtering and sorting; grouping and aggregation; multi-table joins; top-N rankings.
- Missing values and inconsistent categories; ambiguous questions; unsupported questions; empty results.
- Unsafe generated SQL; model offline and timeout; truncation and warnings.

Deliverables

- Working UI, integration document, mock fixtures, evaluation set, end-to-end tests, setup/run instructions, test report and demo script.

Acceptance: A fresh environment can run the full question-to-verified-insight flow on the test database. The demo covers a join/aggregation, a messy-data case and an ambiguous or unsupported question.

7. Shared Integration Contract

Before module implementation is finalized, the team should agree on the shared input/output shapes. Names below are proposed concepts and must be reconciled with the actual repository.

Core interfaces

```
get_schema(database_config) -> dict
profile_database(database_config) -> dict
generate_sql(question, schema_context) -> dict
validate_sql(sql, dialect) -> dict
execute_query(sql, database_config) -> dict
verify_result(question, result) -> dict
run_analysis(question, database_config) -> dict
```

Suggested response envelope

```
{
"status": "success",
"answer": "Explanation grounded in the query result",
"sql": "SELECT ...",
"columns": ["region", "total_sales"],
"rows": [["West", 12000]],
"row_count": 1,
"truncated": false,
"warnings": [],
"verification": {"passed": true},
"chart": {"type": "bar", "x": "region", "y": "total_sales"}
}
```

Use only real values from execution. The example above illustrates the structure, not a result that should be hardcoded. Define consistent statuses, at minimum success, clarification_needed and error. Define how empty results and partial/truncated results are represented.

Responsibility boundaries

- Agent 1 generates SQL and synthesizes language; it does not execute SQL or bypass validation.
- Agent 2 owns schema discovery, profiling and reproducible sample data; it does not silently clean source data.
- Agent 3 owns validation, execution restrictions and result verification.
- Agent 4 owns UI integration, evaluation and end-to-end behaviour; it does not duplicate business logic or bypass the safety layer.

8. Integration Sequence and Test Gates

Gate 1 — Baseline: Confirm repository access, dependencies, startup steps and current tests. Save a known-good baseline.

Gate 2 — Contract: Agree on schema, profiling, generation, execution and response interfaces. Document error/status semantics.

Gate 3 — Independent modules: Each agent works in its assigned module and uses mocks or fixtures for dependencies not yet ready.

Gate 4 — Database first: Integrate schema discovery and the synthetic test database.

Gate 5 — Safety path: Integrate validator, read-only execution and result verification; prove unsafe queries are rejected.

Gate 6 — AI integration: Connect model client and schema-aware SQL generation through the safety path.

Gate 7 — UI integration: Connect the UI to the complete orchestrated workflow and add visible error states.

Gate 8 — Regression: Run module tests and end-to-end tests on a clean environment; resolve failures with the responsible owner.

Gate 9 — Demo readiness: Verify setup documentation, sample data, model configuration, known limitations and demo scenarios.

9. Final Acceptance Checklist

- A clean clone can be installed and started using documented steps.
- The synthetic multi-table database can be created reproducibly.
- Schema discovery identifies relevant tables, columns and relationships.
- Data profiling reports issues without mutating source records.
- Natural-language questions produce schema-grounded SQL.
- All SQL passes the same validator before execution.
- Writes, DDL, multiple statements and unsupported unsafe operations are rejected.
- Execution has appropriate permissions, timeouts and result limits.
- Errors are distinguishable from successful empty results.
- Answer values are traceable to actual query output.
- Tables, charts, warnings and clarification states render correctly.
- Model offline, timeout and invalid-response cases are handled cleanly.
- Unit tests and end-to-end tests pass; known limitations are documented.
- No secrets, local credentials or private production data are committed.

10. Demonstration Plan

Scenario A — Successful analysis: Ask a question requiring a join and aggregation, such as revenue by product category. Show the generated SQL, actual result table, chart and explanation.

Scenario B — Messy data: Ask about category totals when the sample data contains inconsistent category labels or missing values. Show the profiler's warning and explain how it affects interpretation.

Scenario C — Ambiguous or unsupported: Ask a question with an unspecified time period or a metric not present in the schema. The application should ask for clarification or explain the limitation instead of inventing an answer.

Scenario D — Safety: Demonstrate that a write query or multi-statement request is rejected before execution.

Final note

This report defines the intended work allocation and target behaviour. The exact existing source structure, supported database types, UI framework and current implementation must be confirmed by inspecting the repository before coding against specific paths.